

E-Commerce Data Pipeline

Project Documentation

Azure Data Factory • ADLS Gen2 • Databricks • Delta Lake

1. Project Overview

This project builds an end-to-end automated data pipeline for an e-commerce business. Raw operational data — orders, customers, inventory, and order items — is exported as flat CSV files and stored in Azure Data Lake Storage Gen2. The pipeline ingests these files, processes them through a four-layer architecture, and delivers clean business metrics ready for reporting.

The pipeline runs as a Databricks notebook using PySpark and Delta Lake on a Hive Metastore. Azure Data Factory is used to orchestrate the ingestion and trigger the notebook on a scheduled basis.

Problem Being Solved

Without an automated pipeline, teams rely on manual scripts to process these files, which leads to:

- Pipeline failures when source file formats change unexpectedly
- Duplicate or invalid records silently entering reports
- Delayed insights because data is only processed when someone manually runs a script

This pipeline solves all three by automating ingestion, enforcing data quality rules at every stage, and storing data in Delta format with full audit history.

2. Technology Stack

Tool / Service	How It Is Used in This Project
Azure Data Factory	Schedules and triggers the pipeline. Copies CSV files from the source into ADLS Gen2 using Copy Activity. Triggers the Databricks notebook via a Web Activity after ingestion.
Azure Data Lake Storage Gen2	Acts as the raw file landing zone. All CSVs are stored here in the 'landing' container before the pipeline reads them.
Azure Databricks	Runs the PySpark notebook that processes all pipeline layers — Landing, Bronze, Silver, Gold, and Reconciliation.
Apache Spark (PySpark)	Used to read, transform, deduplicate, validate, and write data at every layer of the pipeline.
Delta Lake	Storage format for all tables. Provides ACID transactions, schema enforcement, and time-travel (query historical data).
Hive Metastore	Stores table metadata. Four databases are created here — one per pipeline layer — so all tables are queryable by name.
Databricks Secrets	Stores the Azure storage account key securely. The notebook retrieves it at runtime using <code>dbutils.secrets.get()</code> rather than hardcoding credentials.

3. Data Sources

Four CSV files are used as input. They are synthetically generated to simulate real e-commerce operational data and include intentional data quality issues to test the pipeline's validation logic.

File	Rows	Frequency	Intentional DQ Issues
orders.csv	50,000 + 150 dupes	Every 6 hours	150 duplicate order IDs, ~500 null customer IDs, ~9,000 invalid status values
order_items.csv	200,000	Every 6 hours	~3,000 null quantities, ~2,000 negative unit prices
customers.csv	10,000	Daily (full snapshot)	~200 malformed email addresses
inventory.csv	5,000	Every 6 hours	~150 null stock quantity values

4. Pipeline Architecture

The pipeline follows a Medallion Architecture — a standard industry pattern for data lakes. Data moves through four progressively cleaner layers, each stored as a separate database in Hive Metastore.

ADLS Gen2 (CSV files) → Landing → Bronze → Silver → Gold → Reports

Layer 1 — Landing (hive_metastore.ecommerce_landing)

- Reads CSV files directly from ADLS Gen2 using spark.read with header=true and inferSchema=false
- All columns are kept as STRING — no type interpretation at this stage
- Two metadata columns are added: landing_timestamp (when the file was read) and source_file_name (full ADLS path of the source file)
- Written to Delta tables in APPEND mode so every file load is preserved
- Tables: orders, order_items, customers, inventory

Layer 2 — Bronze (hive_metastore.ecommerce_bronze)

- Reads from the Landing tables and re-writes them with additional metadata
- All source data columns remain STRING — this is intentional to preserve a stable, unmodified copy of the raw data
- Adds bronze_ingestion_timestamp (when the record entered the bronze layer) and load_date (date partition column)
- Tables are partitioned by load_date for efficient querying by date range
- Written as Delta with overwriteSchema enabled to handle any structural changes
- Tables: orders, order_items, customers, inventory

Layer 3 — Silver (hive_metastore.ecommerce_silver)

- This is where all data quality work and type casting happens
- Deduplication: window function (ROW_NUMBER partitioned by primary key, ordered by bronze_ingestion_timestamp DESC) removes duplicate records from source re-exports
- Type casting: all columns are cast from STRING to their proper business types (timestamps, doubles, integers)
- Data quality filters: rows failing critical checks are written to separate quarantine tables with a quarantine_reason column

- Customers use Delta MERGE (SCD Type-1) because the daily snapshot may update existing customer records
- Tables: orders, order_items, customers, inventory, orders_quarantine, order_items_quarantine

Layer 4 — Gold (hive_metastore.ecommerce_gold)

- Pre-computed business aggregates — no raw records, only metrics
- daily_revenue: total revenue, order count, average order value grouped by date, region, and category
- fulfillment_kpi: delivery rate, cancellation rate, shipment rate grouped by date, warehouse, and region
- inventory_health: stock status (healthy / stockout / below_reorder / overstock), reorder flags, and 30-day demand per SKU
- customer_ltv: lifetime spend, order frequency, recency, and segment (VIP / High Value / Mid Value / Low Value) per customer
- Each table prints a KPI summary to the notebook output after writing

Layer 5 — Reconciliation (in gold database)

- reconciliation_row_counts: captures the row count of every table at every layer in a single audit table, allowing verification that no records were lost
- reconciliation_dq_summary: for orders and order_items, shows the bronze count, silver (clean) count, quarantined count, pass rate %, and quarantine rate %
- Both tables are written to the gold database and updated at the end of every pipeline run

5. Hive Metastore Database Structure

The setup cell creates four databases in Hive Metastore — one per layer. The naming convention is `ecommerce_<layer>`.

Database	Tables
hive_metastore.ecommerce_landing	orders, order_items, customers, inventory
hive_metastore.ecommerce_bronze	orders, order_items, customers, inventory
hive_metastore.ecommerce_silver	orders, order_items, customers, inventory, orders_quarantine, order_items_quarantine
hive_metastore.ecommerce_gold	daily_revenue, fulfillment_kpi, inventory_health, customer_ltv, reconciliation_row_counts, reconciliation_dq_summary

6. Data Quality Rules

Data quality checks are applied in the Silver layer. Three types of checks are used, translated from DLT EXPECT constraints into standard PySpark filters:

Table	Check	Condition	Action on Failure
orders	order_id not null	<code>order_id IS NOT NULL</code>	Assert — pipeline aborts immediately
orders	Valid status	<code>status IN ('placed', 'shipped', 'delivered', 'cancelled')</code>	Row sent to orders_quarantine
orders	Positive total	<code>total_amount > 0</code>	Row sent to orders_quarantine

orders	Customer ID not null	customer_id IS NOT NULL	Row sent to orders_quarantine
order_items	Positive quantity	quantity IS NOT NULL AND quantity > 0	Row sent to order_items_quarantine
order_items	Positive unit price	unit_price IS NOT NULL AND unit_price > 0	Row sent to order_items_quarantine
inventory	Stock qty not null	stock_quantity IS NOT NULL	Row dropped from silver
inventory	Non-negative stock	stock_quantity >= 0	Advisory warning only — row kept
customers	Valid email format	email CONTAINS '@'	Logged (not filtered — row kept)

Every rejected row from orders and order_items is written to a quarantine table (orders_quarantine, order_items_quarantine) with a quarantine_reason column explaining why it was rejected. These rows are never deleted and can be corrected and replayed.

7. Key Implementation Details

Storage Access

The storage account key is stored in a Databricks Secret Scope (scope: azure-storage, key: storage-account-key). The notebook retrieves it at runtime and sets it as a Spark config property. This avoids hardcoding any credentials in the notebook code.

```
account_key = dbutils.secrets.get(scope="azure-storage", key="storage-account-key")
spark.conf.set("fs.azure.account.key.<account>.dfs.core.windows.net", account_key)
```

Deduplication

The source system occasionally re-exports the same record in consecutive file drops (e.g., 150 duplicate order IDs in the orders file). Deduplication is applied in the Silver layer using a window function that partitions by the primary key and orders by bronze_ingestion_timestamp descending. Only the most recently ingested version of each record is kept.

```
dedup_window =
Window.partitionBy("order_id").orderBy(F.col("bronze_ingestion_timestamp").desc())
df = df.withColumn("_rn", F.row_number().over(dedup_window)).filter(F.col("_rn") == 1)
```

SCD Type-1 for Customers

Customers arrive as a full daily snapshot, meaning the same customer_id may appear across multiple loads with updated information (e.g., new email, city change). A Delta MERGE operation handles this:

- If the customer already exists in the silver table: update all fields (SCD Type-1 overwrite)
- If the customer is new: insert the record
- On the first pipeline run (empty target table): a full overwrite is used instead of MERGE

Gold KPI Summaries

After each Gold table is written, the notebook computes and prints a KPI summary directly in the cell output. This gives an immediate sanity check after each layer completes. For example:

- Sales: total revenue, total orders, average order value, top region, top category
- Fulfilment: delivery rate %, cancellation rate %, best and worst warehouse by delivery rate
- Inventory: total SKUs, stockout count, below-reorder count, overstock count, total inventory value

- Customer LTV: total customers, active customer %, average LTV, segment breakdown table

8. How to Run the Pipeline

#	Step
1	Upload the four CSV files (orders.csv, order_items.csv, customers.csv, inventory.csv) to the 'landing' container in the ADLS Gen2 storage account hrm6329storageaccount.
2	Ensure the Databricks Secret Scope 'azure-storage' exists and contains the storage account key under the key name 'storage-account-key'.
3	Attach the notebook to a Single User Standard (Legacy) cluster. Run the setup cells (cells 1–5) first to create the Hive Metastore databases.
4	Run the main pipeline cells (cells 6 onwards) in order: Landing → Bronze → Silver → Gold → Reconciliation.
5	Review the printed KPI summaries in each Gold cell output and the reconciliation tables to verify the pipeline completed correctly.
6	ADF triggers this notebook automatically on a 6-hour schedule after the Copy Activity deposits new files into ADLS.

9. Reconciliation and Verification

The final cells of the pipeline produce two audit tables that confirm the pipeline ran correctly.

reconciliation_row_counts

A single table showing the row count of every table at every layer (landing, bronze, silver, gold) with a captured_at timestamp. This allows you to verify that the expected number of records flowed through each stage and that nothing was silently dropped.

reconciliation_dq_summary

For the orders and order_items tables, this shows:

- bronze_row_count: how many rows entered the bronze layer
- silver_row_count: how many passed all DQ checks and entered silver
- quarantined_rows: how many were rejected and sent to quarantine
- pass_rate_pct: percentage of bronze rows that passed DQ
- quarantine_rate_pct: percentage of bronze rows that were quarantined

Both tables are written to the gold database and refreshed on every pipeline run.